Dyer
Innovation

CLAUDE CERTIFIED ARCHITECT · SCENARIO 5

# Code Generation with Claude Code

A study guide for the three-tier instruction model, context isolation via fork, plan-mode discipline, and non-interactive CI execution.

LAST UPDATED

April 27, 2026

THIS GUIDE EMPHASIZES

Three-tier instruction model (CLAUDE.md / rules / skills) · context: fork for sub-agent isolation · Plan mode vs. direct execution · Non-interactive CI mode (the -p flag)

# Contents


01 Exam Context 3

02 Core Architecture: The Claude Code File System 4

2.1 @import Syntax 4
2.2 /memory Command 4

03 The Three-Tier Instruction Model 5

3.1 The Decision Rule 5
3.2 Key Exam Scenario 5

04 Path-Scoped Rules (.claude/rules/) 6

4.1 Glob Patterns 6
4.2 Example Rule File 6
4.3 Rules vs. Subdirectory CLAUDE.md Files 6

05 Skills — On-Demand Task Workflows 8

5.1 Skill Structure 8
5.2 Critical Frontmatter Fields 8
5.3 context: fork — The Isolation Mechanism 8
5.4 allowed-tools — Least-Privilege Tool Access 8
5.5 Skill Scope & Sharing 8

06 Plan Mode vs. Direct Execution 10

6.1 Plan Mode 10
6.2 When to Use Each Mode 10

# 01 Exam Context

This guide covers the **Code Generation with Claude Code** scenario, one of six scenarios on the CCA Foundations exam.

**Domain mapping:**

- **Primary:** Domain 3 — Claude Code Configuration & Workflows (20%)
- **Secondary:** Domain 5 — Context Management & Reliability (15%)
- **Light touch:** Domain 1 — Agentic Patterns (when subagents and context isolation come up)

TIP

### Key Insight

— The exam tests architectural judgment in production scenarios — not trivia

— Every question drops you into a realistic situation

— You must pick the correct configuration mechanism for the problem

— Key skill: knowing which mechanism solves a given problem

# 02 Core Architecture: The Claude Code File System

Understanding the file hierarchy is foundational. Every exam question about "where should this go" maps to this structure:

```
your-project/
├── CLAUDE.md                    ← Always-on project instructions
└── .claude/
    ├── settings.json            ← Permissions, model, hooks
    ├── settings.local.json      ← Personal settings (gitignored)
    ├── rules/                   ← Path-scoped modular instructions
    ├── skills/                  ← On-demand workflows (slash commands)
    ├── commands/                ← Legacy; merged into skills
    └── .mcp.json                ← MCP server integrations

~/.claude/                       ← User-level (all projects)
├── CLAUDE.md                    ← Personal global preferences
├── rules/                       ← Personal rules across all projects
└── skills/                      ← Personal skills across all projects
```

**Priority order (lowest → highest):** Global `~/.claude/CLAUDE.md` → Project `CLAUDE.md` → `.claude/rules/` files → `.claude/settings.local.json`

## @import Syntax

The `@import` syntax lets you reference external files from `CLAUDE.md` to keep it modular.

**Why use it:**

- Pull in personal preferences without cluttering the shared project file
- Selectively include standards relevant to specific packages
- Keep `CLAUDE.md` focused and modular

EXAMPLE

### Example: Importing Personal Preferences

```
# Project CLAUDE.md

## Project Standards
- Use TypeScript strict mode
- Follow ESLint config

## Individual Preferences
- @~/.claude/my-project-instructions.md
```

**First-time import behavior:**

- Claude Code shows an approval dialog listing all external imports
- If you decline, the imports stay disabled
- The dialog does not appear again after the initial decision

EXAMPLE

### Example: Per-Package Standards

```
# packages/api/CLAUDE.md
- @../../standards/api-conventions.md
- @../../standards/error-handling.md

# packages/frontend/CLAUDE.md
- @../../standards/react-patterns.md
- @../../standards/accessibility.md
```

TIP

### Exam scenario

— Monorepo with multiple packages, each maintained by different teams

— Each team has different applicable standards

— Use `@import` so each package's `CLAUDE.md` selectively includes only the relevant standards files

— Avoids dumping everything into one root file

# /memory Command

The `/memory` command shows which memory files (CLAUDE.md files, rules, imports) are currently loaded in your session. It is the primary diagnostic tool when Claude isn't following expected instructions.

## When to use `/memory`

- A new team member reports that Claude isn't following project conventions — run `/memory` to check if instructions are in user-level ( `~/.claude/CLAUDE.md` ) instead of project-level
- Claude behaves inconsistently across sessions — run `/memory` to verify which files are loaded
- After adding new `.claude/rules/` files — confirm they're being picked up

**TIP**

### Exam pattern

— Symptom: "A new team member clones the repo but Claude doesn't follow the project conventions"

— Likely root cause: instructions are in user-level config rather than project-level

— Diagnostic: use `/memory` to verify which files are loaded

# 03 The Three-Tier Instruction Model

**TIP**

**This is the single most important decision framework on the exam.**

Nearly every configuration question boils down to: Is this always-on, path-scoped, or task-specific?

| MECHANISM | WHEN IT LOADS | USE FOR | EXAM SIGNAL |
|---|---|---|---|
| **CLAUDE.md** | Every session, automatically | Universal conventions — coding standards, build commands, testing conventions | "Always follow," "every session," "all developers" |
| **.claude/rules/** | Automatically, scoped by file path globs | Conventions tied to specific file types or directories | "When working on [file type]," conventions tied to file identity |
| **Skills** | On demand, via slash command or auto-invocation | Discrete workflows and tasks | "When performing [task]," task-specific procedures |

## The Decision Rule

**Ask yourself:** Does this guidance apply based on WHAT FILES are being touched, or based on WHAT TASK is being performed?

- **File identity** → Rules (path globs can match it)
- **Task type** → Skills (only a human or Claude can know the intent)
- **Always** → CLAUDE.md

# Key Exam Scenario

**TIP**

### Pattern

- Symptom: "Context X is useful when doing Task Y in Directory Z, but not for other tasks in Directory Z"
- Answer: always a skill
- Why: path-scoped rules can't distinguish tasks within the same directory

# 04 Path-Scoped Rules (.claude/rules/)

Markdown files in `.claude/rules/` that are automatically loaded into Claude's context. They carry the same high priority as `CLAUDE.md` content.

## Glob Patterns

Glob patterns are wildcard-based file matching expressions from Unix systems. They are used in the YAML frontmatter of rule files to control when a rule activates.

| PATTERN | MEANING | EXAMPLE |
|---|---|---|
| `*` | Match anything within one directory level | `*.ts` matches `index.ts` but not `src/index.ts` |
| `**` | Match across any number of directory levels | `**/*.ts` matches `src/api/auth.ts` |
| `?` | Match a single character | `file?.ts` matches `file1.ts` but not `file12.ts` |
| `{}` | List alternatives | `*.{ts,tsx}` matches both `.ts` and `.tsx` files |

## Example Rule File

```
---
paths: **/*.test.ts, **/*.test.tsx
---

# Testing Conventions
- Use describe/it blocks
- Mock external dependencies
```

**TIP**

### Why globs matter

- Globs match file identity (what a file is), not file location (where it lives)
- This distinction matters for co-located test files
- A `CLAUDE.md` in a subdirectory applies to everything there
- `**/*.test.*` targets only test files wherever they are

# Rules vs. Subdirectory CLAUDE.md Files

| FEATURE | `.CLAUDE/RULES/` WITH GLOBS | SUBDIRECTORY `CLAUDE.MD` |
|---|---|---|
| Scoping | By filename pattern | By directory location |
| Co-located test files | Can target separately | Cannot distinguish from source files |
| Loaded when | Claude works on matching files | Claude reads files in that directory |
| Shared via version control | Yes | Yes |

# 05 Skills — On-Demand Task Workflows

Skills are prompt-based workflows stored in `SKILL.md` files that Claude loads when invoked (via `/skill-name` slash command or automatically when relevant). They follow the Agent Skills open standard, extended by Claude Code with additional frontmatter fields.

## Skill Structure

```
.claude/skills/
└── my-skill/
    ├── SKILL.md          ← Required: frontmatter + instructions
    ├── scripts/          ← Optional: executable code
    ├── references/       ← Optional: docs loaded into context
    └── assets/           ← Optional: templates, binary files
```

## Critical Frontmatter Fields

These are the fields the exam tests most heavily:

| FIELD | PURPOSE | EXAM RELEVANCE |
|---|---|---|
| `argument-hint` | Shows expected arguments in command picker (e.g., `[migration name]` ) | Solves missing-arguments problems structurally |
| `context` | `inherit` (default) or `fork` | HIGH — solves context contamination |
| `allowed-tools` | Space-separated list of permitted tools | HIGH — solves overly broad tool access |
| `model` | Override model ( `haiku` , `sonnet` , `opus` ) | Use faster model for specific tasks |
| `agent` | Subagent type when `context: fork` is set (e.g., `Explore` ) | Pairs with fork for specialized execution |
| `disable-model-invocation` | `true` = only user can invoke | Prevents auto-triggering side-effect-heavy skills |
| `user-invocable` | `false` = hidden from slash menu, Claude-only | Background knowledge skills |

# context: fork – The Isolation Mechanism

> **TIP**
>
> This is one of the most-tested concepts in the Code Generation scenario.

When a skill has `context: fork`, it runs in an isolated sub-agent context:

- The skill gets its own context window
- It does NOT inherit conversation history
- Only the results/summary return to the main conversation
- The main conversation's context stays clean

**When to use `context: fork`:**

- The skill generates verbose output (codebase analysis, exploration)
- The skill's intermediate reasoning shouldn't influence later work (brainstorming alternatives)

**When NOT to use `context: fork`:**

- The skill contains only guidelines/conventions (no actionable task for the sub-agent)
- The skill needs access to the current conversation's context

# allowed-tools — Least-Privilege Tool Access

Controls which tools run without permission prompts when the skill is active.

```
# Read-only analysis
allowed-tools: Read Grep Glob

# File modifications
allowed-tools: Read Edit Write

# Git operations
allowed-tools: Read Write Bash(git *)

# MCP tools
allowed-tools: Read mcp__linear__create_issue
```

**TIP**

### Exam pattern

— Symptom: a skill accidentally triggers destructive operations

— Answer: restrict `allowed-tools` to only what the skill needs

— Principle: least-privilege enforcement

# Skill Scope & Sharing

| LOCATION | SCOPE | SHARED VIA GIT? |
|---|---|---|
| `.claude/skills/` (project) | Team — all developers on the project | Yes |
| `~/.claude/skills/` (user) | Personal — all your projects | No |

**TRAP**

### Exam trap

# 06 Plan Mode vs. Direct Execution

## Plan Mode

A read-only operating mode where Claude can analyze the codebase, ask questions, and generate implementation plans — but cannot modify files or run commands.

- **Activate:** `Shift+Tab` twice, or `/plan` command
- **Exit:** `Shift+Tab` again
- **Edit plan directly:** `Ctrl+G`

**Available tools in Plan Mode (read-only):** Read, LS, Glob, Grep, Task, TodoRead/TodoWrite, WebFetch, WebSearch, NotebookRead

**Blocked in Plan Mode:** Edit, MultiEdit, Write, Bash, NotebookEdit, state-modifying MCP tools

## When to Use Each Mode

| SCENARIO | MODE | WHY |
|---|---|---|
| Multi-file restructuring (monolith → microservices) | Plan | Wrong boundaries are expensive to undo |
| Ambiguous requirements with multiple architectural options | Plan | Need to evaluate tradeoffs first |
| Exploring unfamiliar codebase | Plan | Read-only analysis without accidental changes |
| Single-file bug fix with clear cause | Direct | Path is obvious, planning adds overhead |
| Small, well-defined feature addition | Direct | Clear scope, low risk |
| Implementing from an approved plan | Direct | Planning already done |

**TIP**

### Decision Heuristic

- The more files affected, the more valuable plan mode becomes
- The more irreversible the architectural decisions, the more valuable plan mode becomes
- Can you describe the exact diff in one sentence? → Direct execution
- Can't describe the exact diff? → Plan first

# 07 Context Management Patterns

## The Explore Subagent

When a phase of work generates lots of verbose output (discovery, analysis) that isn't needed in the main conversation afterward, delegate it to a subagent.

**How it works:**

- The subagent gets its own context window
- It does the heavy work in isolation
- It returns only a summary to the main conversation

TIP

### Exam pattern

— Symptom: "Phase 1 generates verbose output and context is filling up before Phase 2"

— Answer: use the Explore subagent (or `context: fork` in a skill) to isolate Phase 1

## /compact

Compresses conversation history to free up context space. Useful but lossy — details may be lost. It's a band-aid, not an architectural solution.

TRAP

### Exam trap

— `/compact` is rarely the best answer
— It's typically a distractor
— Real answer is usually context isolation (subagents or `context: fork`)

# Context Contamination

When exploratory or verbose output from one task influences subsequent tasks in the same session.

**Symptoms:**

- Claude references abandoned approaches
- Claude maintains outdated assumptions
- Claude loses track of the original task

**Solution:** Always use `context: fork` for exploratory or analytical skills.

---

# 08 MCP Server Configuration

For team-shared MCP integrations (like a GitHub server), configure in the project's `.mcp.json` with environment variable expansion for credentials:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

**TIP**

### Exam pattern

- Symptom: "Team needs consistent tooling without committing credentials"
- Answer: project-scoped `.mcp.json` with `${ENV_VAR}` expansion
- Config structure is shared via version control
- Secrets are injected via environment variables

# 09 Prompting & Communication Patterns

## Examples Beat Specifications

When Claude keeps misinterpreting prose requirements after multiple iterations, concrete input-output examples are the highest-bandwidth way to communicate intent.

**Hierarchy of Communication Clarity:**

- **Concrete examples** (input → expected output) — unambiguous
- **Precise specifications** (JSON schema, exact field mappings) — structural but doesn't show transformation logic
- **Detailed prose** — still requires interpretation
- **Vague prose** — high ambiguity

## Test-Driven Iteration

Write test suites first, then iterate by sharing test failures to guide Claude toward progressive improvement. This gives Claude concrete, machine-verifiable feedback rather than subjective prose corrections.

**The TDD Loop with Claude:**

1. Write tests covering expected behavior, edge cases, and performance requirements
2. Ask Claude to implement the function
3. Run tests, share the failure output with Claude
4. Claude fixes based on specific test failures
5. Repeat until all tests pass

EXAMPLE

Example:

```
// You write this test first:
test('handles null values in migration', () => {
  const input = { name: null, age: 25, email: null };
  const result = migrateRecord(input);
  expect(result.name).toBe('UNKNOWN');
  expect(result.email).toBeNull();
});

// Then share the failure output with Claude:
// 'Test failed: expected UNKNOWN, received null'
// Claude now has concrete, unambiguous feedback to fix.
```

**TIP**

### Why this works

— Test failures are precise, unambiguous specifications

— Telling Claude "handle nulls properly" is vague

— Showing it "expected UNKNOWN, received null" is exact

# The Interview Pattern

Have Claude ask questions to surface design considerations the developer may not have anticipated before implementing. This is especially valuable in unfamiliar domains where you don't know what you don't know.

**How to Apply:**

```
Prompt: 'I need to add a caching layer to our API.
Before implementing, interview me about the requirements.
Ask about cache invalidation strategies, failure modes,
consistency requirements, and anything else I should
consider.'
```

Claude might surface considerations like:

- What happens when the cache is down?
- Do you need cache-aside or write-through?
- What's the consistency tolerance?
- Should stale data ever be served?

These questions prevent costly rework from missed requirements.

TIP

### Exam pattern

- Use case: implementing in an unfamiliar domain where requirements might be incomplete
- Interview pattern is the right first step — before plan mode, before direct execution
- Purpose: surfaces the unknown unknowns

# Sequential vs. Parallel Issue Resolution

When multiple issues need fixing, the order and grouping of your feedback to Claude matters:

| SITUATION | APPROACH | WHY |
|---|---|---|
| Issues interact with each other (fixing A affects B) | Single detailed message with all issues | Claude can reason about interdependencies |
| Issues are independent (fixing A has no effect on B) | Fix sequentially, one at a time | Each fix gets full attention without distraction |
| Mix of interacting and independent issues | Group interacting issues together; fix groups sequentially | Best of both approaches |

EXAMPLE

### Example:

- Function has a null handling bug AND a separate performance issue in an unrelated code path → fix sequentially
- Null handling bug causes a cascade that also affects the return type validation → describe both in one message so Claude can see the connection

# 10 CI/CD Pipeline Integration

**TIP**

This section covers Task Statement 3.6

— Full exam scenario dedicated to this topic: Scenario 5 — Claude Code for Continuous Integration

— The `-p` flag question appears as a documented sample question

## The -p Flag: Non-Interactive Mode

The `-p` (or `--print`) flag runs Claude Code in non-interactive mode.

**Behavior:**

- Processes the prompt
- Outputs the result to stdout
- Exits without waiting for user input
- Without it, CI jobs hang indefinitely waiting for interactive input

**Basic CI Usage:**

```
# WRONG - hangs waiting for interactive input
claude "Analyze this PR for security issues"

# CORRECT - non-interactive mode
claude -p "Analyze this PR for security issues"
```

**TRAP**

Exam trap — common distractors

— `CLAUDE_HEADLESS=true` — doesn't exist
— `--batch` flag — doesn't exist
— stdin redirect from `/dev/null` — Unix workaround, not the correct approach
— Correct answer: the `-p` flag is the documented, official solution

# Structured CI Output

Use `--output-format json` with `--json-schema` to produce machine-parseable structured findings that automated systems can post as inline PR comments.

```
claude -p \
  --output-format json \
  --json-schema '{"type":"object","properties":{
    "findings":{"type":"array","items":{
      "type":"object","properties":{
        "file":{"type":"string"},
        "line":{"type":"integer"},
        "severity":{"type":"string",
          "enum":["critical","warning","info"]},
        "message":{"type":"string"}
      }
    }}
  }}' \
  "Review this PR for bugs and security issues"
```

**What the JSON output enables:**

- Parse in your CI pipeline to create inline PR comments at the exact file and line where each issue was found
- `--json-schema` is a common D3/D4 integration point for any non-interactive automation
- Compatible automation surfaces: GitHub Actions, GitLab pipelines, pre-commit hooks, webhook handlers
- All benefit from contracting the output shape up front

**Schema as a typed boundary:**

- Treat the schema as a typed boundary between Claude and your downstream parser
- Once the schema is fixed, CI logic can rely on field types, enum values, and required keys
- No defensive parsing needed

# Session Context Isolation for Code Review

RULE

### Key principle

— The same Claude session that generated code is LESS effective at reviewing its own changes

— It retains reasoning context that makes it less likely to question its own decisions

— Use an independent review instance

**Why CI-based code review still matters:**

- Valuable even when the developer used Claude Code to write the code
- CI review instance has no memory of the generation reasoning
- Evaluates the code on its own merits — the same way a human reviewer would

# Avoiding Duplicate Review Comments

When re-running reviews after new commits:

- Include prior review findings in context
- Instruct Claude to report ONLY new or still-unaddressed issues
- Without this, developers see the same comments repeated on every push
- Result: erodes trust in the review system

EXAMPLE

### Example CI Script Pattern:

```
# Fetch prior review comments from the PR
PRIOR_FINDINGS=$(gh pr view $PR --json comments)

# Include them in the review prompt
claude -p \
  "Review this PR. Prior review findings are below.
   Report ONLY new issues or issues that remain
   unaddressed after the latest commits.
   Prior findings: $PRIOR_FINDINGS"
```

# Test Generation in CI

When using Claude Code to generate tests in CI, two practices prevent low-quality output:

- **Provide existing test files in context** — so test generation avoids suggesting duplicate scenarios already covered by the test suite
- **Document testing standards in CLAUDE.md** — include testing conventions, valuable test criteria, and available fixtures so CI-invoked Claude produces high-quality tests consistent with your team's expectations

**EXAMPLE**

Example CLAUDE.md Testing Section for CI:

```
## Testing Standards
- Use Jest with React Testing Library
- Test behavior, not implementation details
- Valuable tests: edge cases, error paths, accessibility
- Low-value tests: trivial getters, pure UI snapshots
- Available fixtures: fixtures/users.ts, fixtures/orders.ts
- Always mock external API calls using msw
- Run: npm test -- --coverage
```

# 11 Exam Decision Patterns – Quick Reference

These are the recurring patterns tested across questions:

| PROBLEM | MECHANISM | WHY |
| --- | --- | --- |
| Conventions that always apply | `CLAUDE.md` | Always loaded |
| Conventions for specific file types | `.claude/rules/` + path globs | Activated by file identity |
| Task-specific workflows | Skills | Activated by intent/invocation |
| Missing required arguments | `argument-hint` frontmatter | Structural prompt at invocation |
| Context contamination from verbose output | `context: fork` | Isolates in sub-agent |
| Overly broad tool access | `allowed-tools` frontmatter | Least-privilege enforcement |
| Architectural ambiguity | Plan mode | Read-only analysis before commitment |
| Team-shared commands | `.claude/commands/` or `.claude/skills/` | Version-controlled, auto-shared |
| Personal workflow customization | `~/.claude/skills/` with distinct name | Personal, no collision |
| Team MCP with individual credentials | `.mcp.json` + `${ENV_VAR}` | Config shared, secrets local |
| Large discovery phase filling context | Explore subagent | Isolates verbose output |
| CLAUDE.md grown too large (400+ lines) | Keep universal in `CLAUDE.md`, tasks to skills | Right tool for each category |
| Modular CLAUDE.md for monorepo | `@import` to pull in per-package standards | Selective inclusion by domain |
| Instructions not loading for a team member | `/memory` command to diagnose | Shows which files are actually loaded |
| Claude misinterprets prose requirements | 2-3 concrete input/output examples | Unambiguous specification |
| Unfamiliar domain with unknown unknowns | Interview pattern before implementing | Surfaces missed requirements |
| Multiple interacting bugs | Single message with all issues | Claude sees interdependencies |

# 12 Key Terminology

| TERM | DEFINITION |
| --- | --- |
| `CLAUDE.md` | Markdown file auto-loaded at session start containing project instructions. Lives at project root or `.claude/CLAUDE.md`. |
| `@import` | Syntax in `CLAUDE.md` for referencing external files (e.g., `@~/.claude/my-rules.md`). Keeps `CLAUDE.md` modular. First-time imports show an approval dialog. |
| `/memory` | Command that shows which memory files are currently loaded. Primary diagnostic tool for inconsistent behavior or missing instructions. |
| Glob pattern | Wildcard file matching syntax (`*`, `**`, `?`, `{}`) used in rule frontmatter to scope rules to specific file paths. |
| Frontmatter | YAML metadata block at the top of a `.md` file (between `---` delimiters) that configures behavior. |
| `context: fork` | Skill frontmatter option that runs the skill in an isolated sub-agent context, preventing context contamination. |
| `context: inherit` | Default. Skill runs in the main conversation context with access to history. |
| `allowed-tools` | Skill frontmatter field listing tools that can run without permission prompts. Enforces least-privilege. |
| `argument-hint` | Skill frontmatter field that shows expected arguments in the command picker. |
| `disable-model-invocation` | Skill frontmatter flag. When true, only the user can invoke the skill — Claude won't auto-trigger it. |
| `$ARGUMENTS` | Variable in skill body containing everything typed after the slash command. |
| Plan mode | Read-only operating mode. Claude can analyze and plan but cannot modify files or run commands. |
| Direct execution | Default mode. Claude can read, write, edit, and run commands. |

# 13 Common Anti-Patterns (Wrong Answers on the Exam)

The exam frequently offers these as plausible-but-incorrect distractors:

| ANTI-PATTERN | WHY IT'S WRONG |
|---|---|
| "Add instructions to ignore prior context" | Soft guardrail. Claude may not follow. Use `context: fork` for structural isolation. |
| "Put everything in `CLAUDE.md`" | Causes priority saturation. Split into rules and skills by scope. |
| "Use `/compact` to manage context" | Lossy band-aid. Use subagents or `context: fork` for structural isolation. |
| "Split into smaller skills to reduce output" | Running three smaller skills in the same session produces the same cumulative context impact. |
| "Switch to a faster model" | Solves speed problems, not context contamination problems. |
| "Add a description warning about destructive ops" | Documentation, not enforcement. Use `allowed-tools` to restrict access. |
| "Username-based conditional logic" | Pollutes team configuration with personal preferences. |
| "Override team skill with same-name personal skill" | Creates invisible behavior differences. Use a distinct name instead. |
| "`CLAUDE_HEADLESS=true` or `--batch` for CI" | These flags don't exist. The correct flag is `-p` (or `--print`) for non-interactive mode. |
| "Same session reviews its own code" | Self-review is unreliable — the model retains reasoning context and won't question its own decisions. Use an independent instance. |
| "Run CI review without prior findings context" | Produces duplicate comments on every push, eroding developer trust. Include prior findings and instruct to report only new issues. |

# 14 Recommended Study Resources

- **Official Exam Guide (PDF)** — S3 hosted via Anthropic Academy
- **Claude Code Memory Docs** — code.claude.com/docs/en/memory
- **Claude Code Skills Docs** — code.claude.com/docs/en/skills
- **Claude Code How It Works** — code.claude.com/docs/en/how-claude-code-works
- **Anthropic Academy (free courses)** — anthropic.skilljar.com
- **Official Practice Test** — 60-question practice exam via Anthropic Academy

Last updated: March 2026 — Claude Certified Architect Foundations (Domain 3 focus)

You've written a function. Now we're going to make it fail on purpose, and then fix it. This is how you learn to debug.